

Customer Churn

Data Science Exercise Report

Malika Patel

Contents

Data checks & preparation.....	3
Assumptions	3
Key Actions	3
Part 1: Product Analytics	5
Assumptions	5
Summary of Findings	5
Limitations.....	8
Recommendations	8
Questions for Further Analysis.....	8
Data Needed for Further Analysis	9
Part 2: Predictive Modeling.....	9
Training Data Preparation	9
Feature Engineering.....	10
Model Selection	12
Evaluation	12
Next Steps:	13

Data checks & preparation

Assumptions

For this exercise, I assumed that the data was clean, as the exercise did not mention the need to clean the data. Nevertheless, I still performed some standard data quality checks so I could be confident that I was performing the subsequent analysis on clean data.

Please refer to the Data Checks & Preparation section in the Jupyter notebook for all checks, as this section will only report important findings or changes.

Key Actions

1. A duplicate customer_id (120) was identified in the app_downloads_df. A review of all provided datasets confirmed that this was the only duplicate customer ID across the data.

Investigation of the duplicate records showed that all values were identical except for the app_download_date, which differed between the two entries. To determine which record was valid, the customer's event timeline was examined. The timeline began after the earlier download date and continued uninterrupted beyond the later download date, providing no evidence of a second app installation.

As the app_downloads_df is intended to record the date a customer first downloaded the app, the earlier download date was considered the correct value. Consequently, the duplicate record with app_download_date = 2022-07-20 was removed, retaining the earliest download date for customer_id 120.

Check of what the duplicate values are from app downloads

	customer_id	app_download_date
191	120	2022-01-13
2462	120	2022-07-20

Check of customers app usage timeline

	timestamp	customer_id	event_name	app_version
33	2022-01-17 11:06:00	120	register_ev_click	1.4
51	2022-01-23 04:36:27	120	submit_meter_reading	1.4
828	2022-04-02 08:21:46	120	view_account_balance	1.4
1275	2022-04-29 15:09:10	120	submit_meter_reading	1.4
2639	2022-06-30 18:01:21	120	submit_meter_reading	1.4
5505	2022-10-21 02:43:23	120	view_account_balance	1.5
7333	2023-01-01 20:12:14	120	view_account_balance	1.5
9612	2023-03-30 09:21:36	120	submit_meter_reading	1.5
10236	2023-04-22 16:54:26	120	submit_meter_reading	1.5

2. The `account_closed_date`, `app_download_date`, and `timestamp` variables were converted from string to datetime format using `pd.to_datetime()` to ensure consistent date formatting and support time-based analyses.
3. Two merged data frames were created:
 - a. The `customer_level_df` combines customer profile, app download, and account closure information, resulting in one row per customer. This dataset was created to support analysis of customer characteristics and account-level outcomes.
 - b. The `customer_event_level_df` combines app event data with customer profile, app download, and account closure information, resulting in one row per recorded event. This dataset was created to support analysis of customer behaviour and event activity over time.

Part 1: Product Analytics

Please refer to Part 1 in the Jupyter Notebook file for the accompanying code.

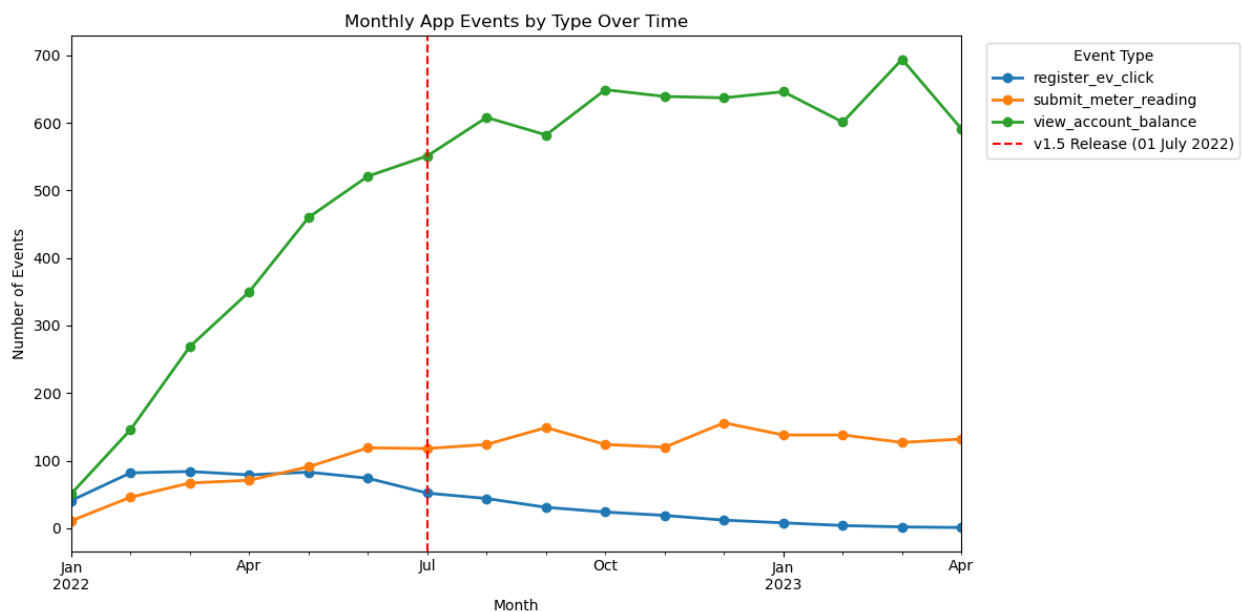
Assumptions

Most people would register their EV soon after downloading the app.

Summary of Findings

Bottom line: The drop in "Register your EV" clicks is real, but the evidence does not support the v1.5 button placement as the primary cause. The decline appears to be driven mainly by a steady fall in new app downloads.

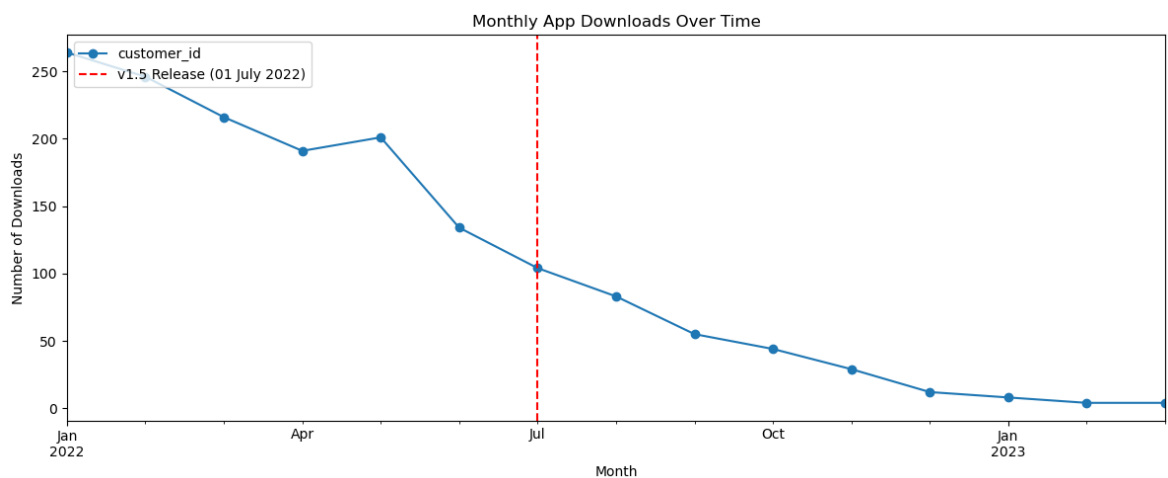
Monthly EV registration clicks declined over 2022–2023 (see Graph 1), and this decline was already visible before the v1.5 release date.



Graph 1 Monthly App Events by Type Over Time

However, monthly app downloads (see Graph 2) show a steady decline across the entire period, starting from January 2022 and well before the v1.5 release on 1 July. This is not a

sudden break around the release; it is a pre-existing trend.

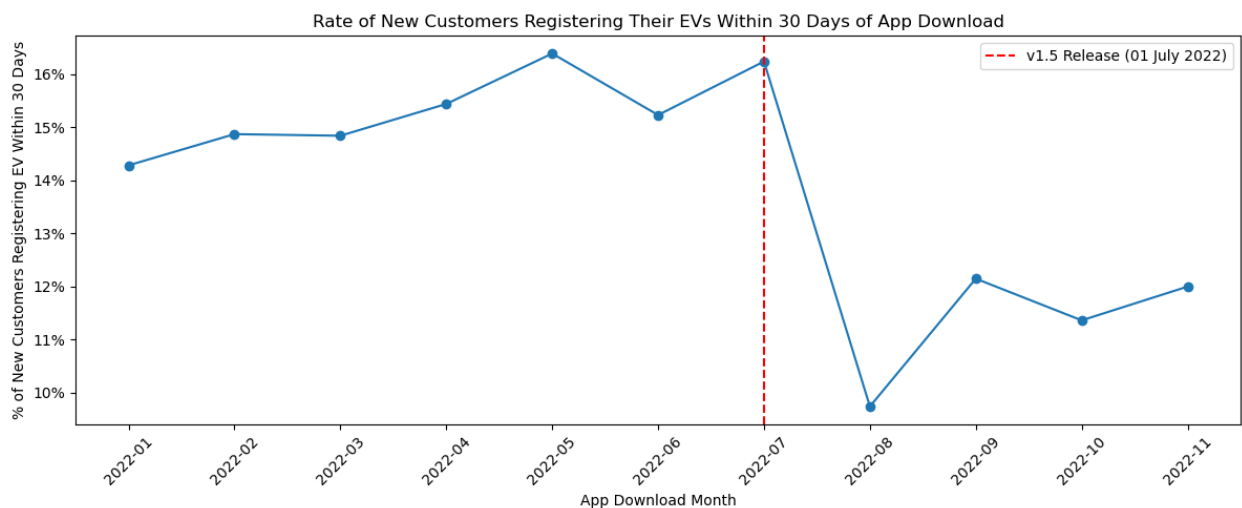


Graph 2 Monthly App Downloads Over Time

EV registration tends to happen early in a customer's app usage, typically soon after download. Therefore, fewer downloads naturally produce fewer raw registration clicks, independent of any button placement change.

To separate this "fewer people to click" effect from a genuine change in behaviour, and to make a fair comparison across cohorts of different sizes and different amounts of time to convert, we looked at the conversion rate within a fixed window: the percentage of each month's new customers who registered an EV within 30 and 60 days of downloading.

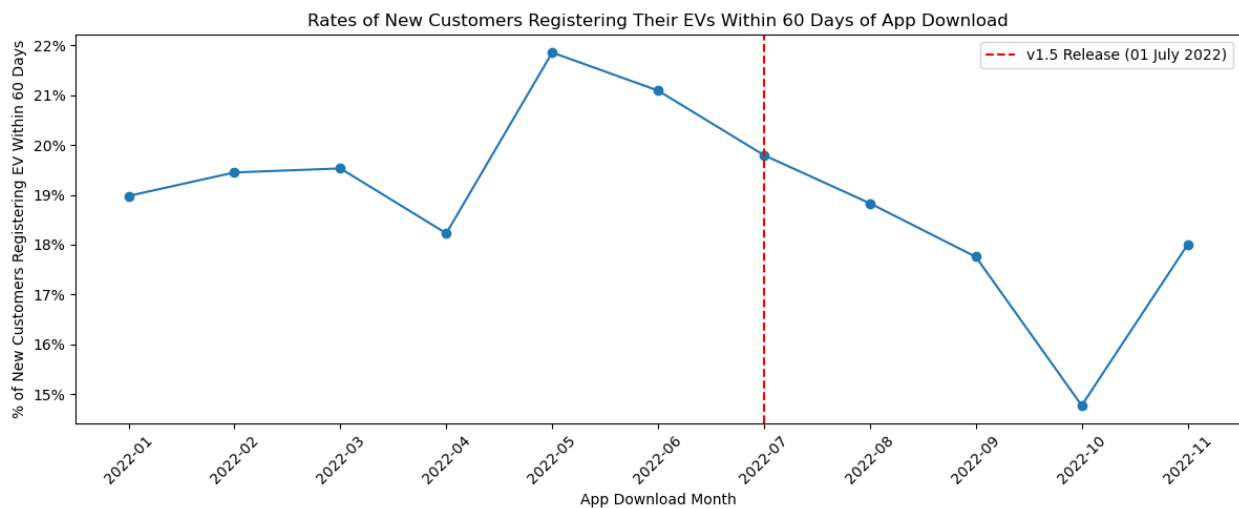
In Graph 3: 30-Day Conversion Rate, conversion rises from 14.3% (January 2022) to a peak of 16.4% (May 2022), dips slightly to 15.2% in June, then recovers to 16.2% in July, the release month itself. It then falls to 9.7% in August, before partially recovering to 11–12% through September–October.



Graph 3 Rates of New Customers Registering Their EVs Within 30 Days of App Download

This is worth being precise about: the shape is not a steady decline from May through October; it is flat-to-rising through July, followed by a sharp single-month drop in August. That distinction matters because if placement were the cause, we would expect the first cohort to only ever see the new button (July downloaders) to convert worse immediately, but July still converts at 16.2%, in line with pre-release months. The crash appears one month later instead, in the August cohort.

The 60-Day Conversion Rate was reviewed as a robustness check, using a longer window to register. It shows a milder version of the same general shape, with a decline around the same period before partially recovering, though the dip is less severe than in the 30-day view.

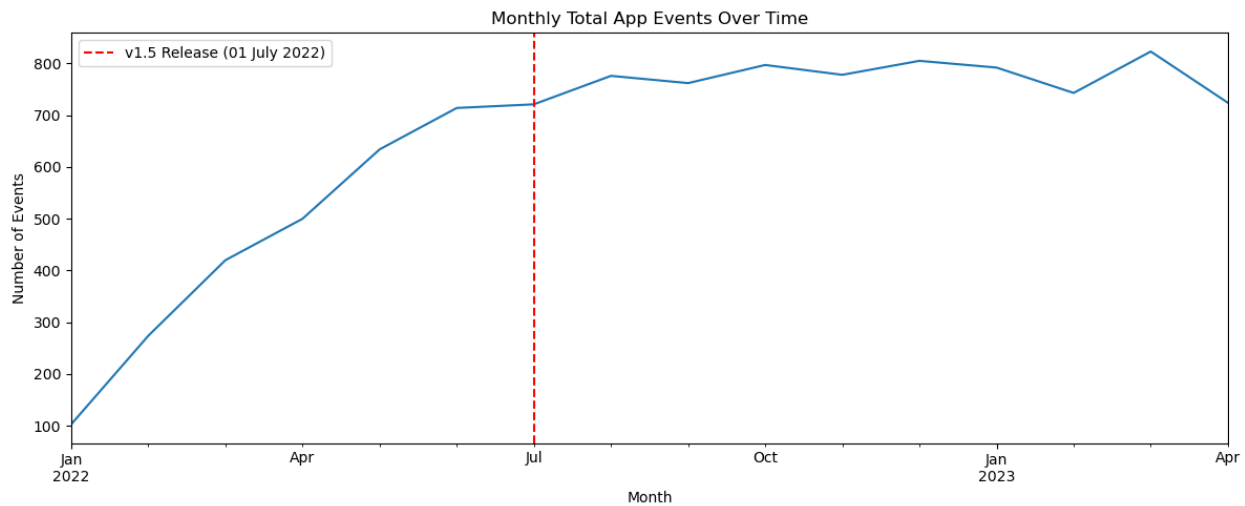


Graph 4 Rates of New Customers Registering Their EVs Within 60 Days of App Download

Other event types (view_account_balance, submit_meter_reading, see Graph 2) remained stable or slightly increased after the release.

Total app events across the whole app (see Graph 5) continued trending upward over the same period, supporting the idea that general engagement was not declining. This appears

specific to EV registration rather than a wider usability issue introduced by v1.5.



Graph 5 Rates of New Customers Registering Their EVs Within 60 Days of App Download

Taken together, since the decline in downloads predates the release by several months, and the sharpest conversion drop occurs in August rather than July, I am fairly confident there is another underlying factor, most likely the drop in app downloads, rather than the button placement itself.

Limitations

Sample sizes shrink sharply towards the end of the dataset (e.g., 24 new customers in January 2023, down to 5 in February 2023), so late-period conversion percentages are noisy and should not be over-interpreted.

Recommendations

Do not change the location of the button yet. The evidence provided is not sufficient to clearly implicate placement.

Start investigating the reason for the decline in app downloads, as this appears to be a bigger driver of the decline in EV click numbers.

If the product manager still wants to investigate whether the button location is the problem, create an experiment where there is a control group (one group sees v1.5, the current version) and another group sees the original design. Customers are randomly assigned into one of these groups so they experience the same or similar external conditions at the same time, rather than relying on a before-and-after comparison. Subsequent analysis can then be performed.

Questions for Further Analysis

What is driving the ongoing decline in new app downloads? (e.g., seasonality, lack of marketing on energy suppliers' behalf, etc.)

I would want to look more closely at customer profiles to see whether features there (tariff type, etc.) are having an impact on the decline.

What happened in August 2022 to cause such a sharp dip (referring to Graph 3)? Would knowing this help explain why there was a one-month delay in terms of the expected dip in rates?

Are the costs from energy suppliers impacting whether customers choose to purchase EVs and therefore have no EV to register?

Data Needed for Further Analysis

Data on how energy suppliers are making their customers aware of their product (e.g., an energy supplier emails its customers about the app, and we receive that information, plus whether they clicked the link and whether they ended up registering).

Customer profile data (already acquired). Additional information such as customers' income, where they live, and whether they have dependents (all factors that may influence how much a customer is willing to pay for an EV).

Energy supplier cost/pricing information.

Logs of which app version each customer was running and when they updated to v1.5, since staggered or delayed updates could mean the July cohort was not fully exposed to the new placement yet. This may explain why the dip appears in August instead.

Part 2: Predictive Modeling

Please refer to Part 2 in the Jupyter Notebook file for the accompanying code.

Training Data Preparation

The objective of the predictive model was to identify customers who were likely to close their accounts. As the prediction target was defined at the customer level, the modelling dataset was constructed so that each row represented a single customer. Customer profile information was combined with aggregated summaries of each customer's app activity, such as the total number of events and counts of specific event types, to produce one record per customer.

This approach was chosen instead of using one row per app event (or one row per customer per month) for two reasons. First, it ensured that each observation corresponded directly to the prediction target. Second, using event-level data would result in repeated customer profile information across multiple rows and introduce dependence between observations from the same customer. If these observations were then randomly divided between the training and validation sets, information from the same customer could appear in both datasets, resulting in an overly optimistic estimate of model performance. Aggregating the events to the customer level avoids this issue while providing a concise summary of each customer's engagement history.

To prevent data leakage, a fixed prediction cutoff date of 19 July 2022 was selected. Only customer profile information and app events that occurred on or before this date were used to construct the model features. Customers who had already closed their accounts on or before the cutoff date were excluded, as they could not subsequently churn within the prediction window. The target variable was then defined as whether a customer closed their account within the four months following the cutoff date.

This approach ensures that the target variable is always determined after the model features have been created. If information occurring after the prediction date were included when constructing the features, the model would have access to future information. For example, customers who had already closed their accounts would have no subsequent app activity, making the prediction artificially easier and leading to data leakage.

The cutoff date and prediction window were selected by considering the available data. The dataset contained 366 account closures across approximately 3,000 customers, meaning that many closures would be excluded depending on the chosen cutoff date. Several cutoff dates and prediction windows were explored to balance two competing objectives: retaining sufficient historical app activity to construct meaningful features while also preserving enough churn events for the model to learn from. A cutoff date of 19 July 2022 with a four-month prediction window provided approximately six months of historical event data and retained a suitable number of churn events for model training.

The remaining observations were then used to construct a separate validation dataset using a later cutoff date. This ensured that the training and validation datasets represented different time periods while following the same feature engineering and target construction process.

Feature Engineering

Features created:

tenure_days

- Number of days between the app download date and the prediction cutoff date
- Captures how long the customer has been using the app, as newer and long-term customers may exhibit different churn behaviour.

days_since_last_event

- Number of days since the customer's most recent app event before the cutoff date.
- Measures recent engagement, with longer periods of inactivity potentially indicating an increased risk of churn.

days_to_first_event

- Number of days between downloading the app and the customer's first recorded event.
- Measures how quickly a customer began engaging with the app after downloading it, which may reflect early adoption and engagement.

has_app_activity

- Binary indicator of whether the customer had any recorded app events before the cutoff date.
- Distinguishes customers who have never used the app from those who have interacted with it at least once.

has_registered_ev

- Binary indicator of whether the customer ever used the register_ev_click feature before the cutoff date.
- Directly captures EV feature engagement, allowing the hypothesis that EV users are less likely to churn to be evaluated.

total_events

- Total count of all app events (of any type) recorded for the customer before the cutoff date.
- Measure of how actively engaged the customer is with the app since low activity may signal disengagement and higher churn risk.

unique_event_types

- Number of distinct event types (out of the app's event types) the customer has triggered before the cutoff date.
- A customer who only ever does one thing (e.g. checks their balance) may behave differently to one who uses several features.

Model Selection

A logistic regression model was selected as the initial predictive model because the objective was a binary classification task: predicting whether a customer would close their account within the defined prediction window.

Logistic regression was chosen because it provides a strong baseline model while maintaining a high level of interpretability. Unlike more complex models that may produce accurate predictions but are difficult to explain, logistic regression allows the relationship between customer characteristics and churn risk to be examined through model coefficients. This is valuable in a retention setting, as it provides insight into which customer behaviours and characteristics are associated with a higher likelihood of churn.

Additionally, logistic regression is suitable for the structure of the dataset, as it can incorporate both continuous and categorical predictors. Continuous variables, such as estimated annual energy usage and customer engagement metrics, can be directly included after standardisation, while categorical variables, such as tariff type and payment method, can be incorporated through one-hot encoding.

Given the relatively small dataset size and the importance of understanding the factors contributing to churn predictions, logistic regression was considered an appropriate model.

Evaluation

As mentioned earlier, a separate validation set was created using data that was not used to train the model. This ensures the model is evaluated on customers it has not previously seen, giving a more honest picture of how it would perform on new, real-world data.

Another important consideration is the class imbalance between customers who remain with the supplier (the majority) and those who close their account (the minority). Because of this imbalance, a model that simply predicted "no churn" for every customer would still achieve a high accuracy score, despite failing to identify customers at risk of leaving. This is

why accuracy alone is not a reliable metric here, and why metrics such as precision, recall, and ROC-AUC were also considered.

Instead, performance was evaluated using three measures: ROC-AUC, precision, and recall. ROC-AUC gives an overall sense of how well the model can distinguish between customers who are likely to churn and those who are not. Precision and recall then assess the model's real-world usefulness. Precision asks: when the model flags a customer as likely to churn, how often is it actually correct? This matters because if precision is low, the business would end up spending time and money on retention efforts for customers who were never going to leave. Recall asks: out of all the customers who did churn, how many did the model successfully identify? This matters because if recall is low, the model is missing the very customers that a retention campaign is intended to reach.

The final model achieved a ROC-AUC score of 0.76, which means it was meaningfully better than random guessing at distinguishing between customers who were more likely to churn and those who were likely to stay. Looking at how it performed in practice: out of the 48 customers who actually churned during the validation period, the model correctly flagged 12 of them (a recall of 25%). On the other hand, because churn is relatively rare overall, many of the customers the model flagged as high risk did not end up churning. Of the customers the model flagged, only 10% actually churned (a precision of 10%). This suggests the model has some genuine signal to build on, but in its current form, it would need refinement before being relied upon to directly target retention efforts, since it currently misses most churners and also over-flags many low-risk customers.

Bottom line: while the model is not precise enough to pinpoint exact churners, it can still help prioritise retention activity by identifying a group of customers who are more likely to be at risk, rather than treating every customer the same. The appropriate threshold to use going forward depends on the business's priorities, such as whether it is more important to identify as many potential churners as possible or to avoid spending on outreach to customers who were not actually going to leave.

Next Steps:

Additional data

- Energy costs/ customer bills
 - Monthly bills or the effective price per kWh would allow the model to distinguish between customers with similar energy usage but very different costs. Customers experiencing higher or rapidly increasing bills may be more likely to churn.
- Customer location

- Regional differences in energy prices, competition, weather, and EV charging infrastructure could all influence customer behaviour and churn risk.
- Customer service interactions
 - Information such as complaints, support tickets, or call centre interactions could provide early indicators of customer dissatisfaction.
- Customer contract data/info
 - Contract start date, renewal date, or remaining contract length because customers approaching the end of a fixed tariff may be more likely to switch suppliers.

Model improvements

- Compare additional models
 - Logistic regression provides a good baseline, but it would be interesting to see how it compares to other models. Especially ones that may be able to capture non linear patterns within customer behavior, such as random forest or gradient boosting.
- Further Hyper parameter tuning.
 - Investigate whether different regularization techniques could help identify informative features
 - Use cross validation to further optimise the performance of the model
- Address class imbalance
 - Use an approach like SMOTE or research other methods that would be appropriate to help address the imbalance
- Thresholds
 - If given more context from the business about certain objectives, a threshold could be set to help meet them (e.g., if the business wanted to prioritise the cost of campaigns, the classification threshold could be adjusted to target only the customers with the highest predicted churn risk)